Calango Packaging Guide

Building the macOS `.dmg` and Linux `.deb` installers

Seixas Research

July 2026 — Calango 26.7.13

Abstract

Calango ships as two native installers built with CMake/CPack: a drag-and-drop disk image (`.dmg`) for macOS and a Debian package (`.deb`) for Ubuntu/Debian. This guide covers the prerequisites, the exact commands, what the packaging pipeline does under the hood, how to verify the artifacts, and the failure modes we have already hit — with their fixes. Everything described here lives in `CMakeLists.txt` (install and CPack sections) and in the `packaging/` directory of the repository.

Contents

---

1 Overview

Both installers come out of the standard CMake flow: *configure* → *build* → *cpack*. The platform decides the generator:

Platform	CPack generator	Artifact
macOS (`-DCALANGO_MACOS_BUNDLE=ON`)	`DragNDrop`	`calango-26.7.13-macos-arm64.dmg`
Linux	`DEB`	`calango_26.7.13_amd64.deb`

Each artifact is accompanied by a `.sha256` checksum file. The package version comes from `project(calango VERSION ...)` in `CMakeLists.txt`, which should match the plain-text `version` file at the repository root.

The supporting files are:

- `packaging/macos/make_icns.sh` — renders `calango.icns` from the master PNG icon in `assets/calango/` (used when no committed `.icns` exists).
- `packaging/macos/embed_python_framework.sh` — makes the `.app` independent of the build machine's Python (Section 4).
- `packaging/linux/calango.desktop` — application launcher entry.
- `packaging/linux/calango-mime.xml` — registers the `application/x-calango-project` MIME type for `.calproj` project files.
- `packaging/linux/postinst`, `postrm` — refresh the desktop/MIME/icon caches on install and removal.

2 macOS: building the `.dmg`

2.1 Prerequisites

- Xcode command-line tools (provides `sips`, `iconutil`, `codesign`, `hdiutil`).
- Qt 6 with the Svg module; Homebrew's `brew install qt` puts it at `/opt/homebrew/opt/qt`.
- CMake $\geq$ 3.21.
- A Python virtualenv with ASE installed, e.g. the project `.venv` (`python3 -m venv .venv && .venv/bin/pip install ase`).

2.2 Commands

From the repository root:

```
cmake -S . -B build-dmg \
  -DCMAKE_BUILD_TYPE=Release \
  -DPython3_EXECUTABLE=$PWD/.venv/bin/python \
  -DCMAKE_PREFIX_PATH=/opt/homebrew/opt/qt \
  -DCALANGO_MACOS_BUNDLE=ON

cmake --build build-dmg -j 8

cd build-dmg && cpack
```

The result is `build-dmg/calango-26.7.13-macos-arm64.dmg`. Mounting it shows `calango.app` next to an `/Applications` symlink — the standard drag-to-install layout. There is deliberately no click-through license sheet (`CPACK_DMG_SLA_USE_RESOURCE_FILE_LICENSE OFF`): a license SLA blocks scripted `hdiutil attach` and adds friction for an MIT-licensed application.

2.3 What the install step does

`cpack` internally runs the CMake install rules into a staging directory (`_CPack_Packages/`) and then wraps the result in a disk image. The install rules perform, in order:

1. **Stage the bundle.** `calango.app` is installed with the Finder icon (`calango.icns`, committed or rendered at build time by `make_icns.sh`) and the `Info.plist` metadata (bundle id `org.seixasresearch.calango`).

2. **Optionally embed a Python environment** into `Contents/Resources/python`, when the cache variable `CALANGO_EMBEDDED_PYTHON_DIR` is set (Section 4).

3. **Deploy Qt with `macdeployqt` — twice.** The tool copies the Qt frameworks and plugins into the bundle and rewrites their load paths to `@executable_path`. Two passes with `-libpath=/opt/homebrew/opt/qt/lib` are required: frameworks copied into the bundle lose the `@loader_path` context their `@rpath` dependencies resolve against, so transitive dependencies (e.g. QtGui → QtDBus, plugin dependencies) are only found when the second pass re-scans the already-copied frameworks. The first pass prints a wall of `ERROR: Cannot resolve rpath` lines — this is expected noise; judge success by the verification steps below.

4. **Embed the linked `Python.framework`.** `embed_python_framework.sh` copies the framework version the binary links against (typically Homebrew's) into `Contents/Frameworks`, replaces Homebrew's dangling `site-packages` symlink with a real directory, recreates the canonical framework symlinks, retargets the load command with `install_name_tool`, and finally re-signs the whole bundle ad hoc (`codesign --force --deep --sign -`). Without the re-sign, Apple-silicon macOS refuses to load the modified binaries.

2.4 Verification

```
APP=_CPack_Packages/Darwin/DragNDrop/calango-26.7.13-macos-arm64/calango.app

# 1. Signature must verify strictly, including nested code
codesign --verify --deep --strict "$APP"

# 2. The packaged binary must start and find a Python with ASE
"$APP/Contents/MacOS/calango" --probe-python

# 3. The image must mount without an SLA prompt
hdiutil attach -readonly -nobrowse calango-26.7.13-macos-arm64.dmg
```

To rebuild the package from a clean state, clear the staging area first with `cmake -E rm -rf _CPack_Packages` — stale staging can mask deployment bugs.

2.5 Troubleshooting

Symptom	Cause / fix
`ERROR: Cannot resolve rpath "@rpath/Qt..."` during `cpack`	Expected noise from `macdeployqt`'s first pass. Only investigate if verification fails afterwards.
`dyld: Library not loaded: @rpath/QtDBus...` when launching the packaged app	A framework is missing from the bundle — the second `macdeployqt` pass or the `-libpath` argument was removed, or the install rpath was stripped. Both fixes live in the `install(CODE ...)` block of `CMakeLists.txt`.
`codesign` reports `No such file or directory` for a path that exists	A dangling symlink inside the bundle (Homebrew's `site-packages` link is the known case, handled by `embed_python_framework.sh`). Find offenders with `find app -type l ! -exec test -e {} \; -print`.
`hdiutil: attach canceled`	The DMG carries a click-through SLA. Keep `CPACK_DMG_SLA_USE_RESOURCE_FILE_LICENSE` set to `OFF`.
`Permission denied` running a `packaging/` script	Cloud-synced checkouts (Dropbox) strip executable bits. CMake therefore invokes every packaging script via `bash <script>`; keep that pattern when adding new ones.

3 Linux: building the `.deb`

3.1 Prerequisites (Ubuntu/Debian)

```
sudo apt install build-essential cmake ninja-build \
  qt6-base-dev libqt6svg6-dev libqt6opengl6-dev \
  libgl1-mesa-dev python3-dev python3-venv \
  dpkg-dev file

python3 -m venv .venv && .venv/bin/pip install ase
```

`dpkg-dev` provides `dpkg-shlibdeps`, which CPack uses to compute the runtime `Depends:` line automatically (`CPACK_DEBIAN_PACKAGE_SHLIBDEPS ON`).

3.2 Commands

```
cmake -S . -B build-deb \
  -DCMAKE_BUILD_TYPE=Release \
  -DPython3_EXECUTABLE=$PWD/.venv/bin/python

cmake --build build-deb -j$(nproc)

cd build-deb && cpack
```

The result is `calango_26.7.13_<arch>.deb`. Install it via `apt` rather than `dpkg -i`, so the dependencies are resolved:

```
sudo apt install ./calango_26.7.13_amd64.deb
```

3.3 Package contents and desktop integration

Path in package	Purpose
`/usr/bin/calango`	The application binary
`/usr/share/applications/calango.desktop`	Launcher entry (`Exec=calango %F`)
`/usr/share/mime/packages/calango-mime.xml`	MIME type for `.calproj`
`/usr/share/icons/hicolor/scalable/apps/calango.svg`	Scalable icon
`/usr/share/pixmaps/calango.png`	Legacy raster icon
`/usr/lib/calango/python/`	Embedded Python (optional, see Section 4)

The MIME registration means double-clicking a `.calproj` workspace in the file manager opens it in Calango; `MainWindow::loadFile` routes `.calproj` arguments to the project loader rather than the ASE structure reader. The `postinst`/`postrm` maintainer scripts refresh `update-mime-database`, `update-desktop-database` and `gtk-update-icon-cache` so the association is live immediately after install (modern dpkg triggers usually do this anyway; the scripts make it robust on minimal systems).

Unless an embedded Python is bundled, the package declares `Depends: python3 (>= 3.9)` and `Recommends: python3-ase`. Without ASE the GUI starts but structure I/O and job submission are disabled; users can also point Calango at any interpreter via `CALANGO_PYTHON`.

3.4 Verification

```
dpkg-deb --info  calango_26.7.13_amd64.deb   # control fields, Depends
dpkg-deb --contents calango_26.7.13_amd64.deb

# after installing:
calango --probe-python
xdg-mime query default application/x-calango-project
```

Status note: the DEB configuration is complete but has so far been validated only structurally (the development machine is a Mac without `dpkg`). The first CI or VM build on Ubuntu should run the verification steps above.

4 The embedded Python environment

Calango embeds CPython at runtime. The interpreter is resolved in this order (documented in `src/python_bridge/PythonEngine.hpp`):

1. `$CALANGO_PYTHON` — explicit interpreter path;
2. `$VIRTUAL_ENV/bin/python` — active virtualenv;
3. a Python bundled by the installers: `Contents/Resources/python/bin/python3` (macOS) or `<prefix>/lib/calango/python/bin/python3` (Linux);
4. the interpreter CMake was configured against (`CALANGO_DEFAULT_PYTHON`, meaningful only on the build machine).

To ship a fully self-contained installer, pass a *relocatable* Python distribution that has ASE installed:

```
cmake -S . -B build-dmg ... \
  -DCALANGO_EMBEDDED_PYTHON_DIR=/path/to/standalone-python
```

A distribution from the `python-build-standalone` project is the recommended source. A plain project `.venv` is *not* relocatable — its `bin/python` and `pyvenv.cfg` refer back to the build machine's base interpreter.

Independent of this option, the macOS pipeline always embeds the `Python.framework` that the binary itself links against (`libpython` via `Development.Embed`), so the packaged app launches on machines without Homebrew. For the interpreter to also find ASE there, either bundle an environment as above or let users install ASE and set `CALANGO_PYTHON`.

5 Known limitations and next steps

- **Code signing:** the `.app` is signed ad hoc. Distribution outside a lab requires a Developer ID certificate and notarization (`codesign -sign "Developer ID..."`, `notarytool submit`, `stapler`); the ad-hoc step in `embed_python_framework.sh` is the place to swap the identity in.
- **DEB validation:** run the Linux pipeline once in CI (Ubuntu 22.04/24.04) and consider `lintian` for policy checks.
- **Full ASE bundling:** wiring a `python-build-standalone` + ASE tree into both installers (and building against its `libpython`) would remove the last runtime dependency on a user-provided Python.